Distributed Systems
CS6421
Performance

Prof. Roozbeh Haghnazar

Slides Credit:

Prof. Tim Wood and Prof. Roozbeh Haghnazar

Final Project

Questions?

- Implementation phase!
  - Get coding!
  - Keep your code in GitHub
- Schedule meetings with us!
  - Especially if you realize you can't achieve what you originally planned
- Timeline
  - Milestone 0: Form a Team
  - Milestone 1: Select a Topic
  - Milestone 2: Literature Survey
  - Milestone 3: Design Document
  - **Milestone 4: Final Presentation**

Last Time…

- Replication and Consistency
  - Why replicate
  - What is consistency?
  - Consistency Models
  - Quorum Replication
- Exam
  - Avg: 90%

This Time…

- Performance in Dist. Systems
  - Introduction
  - Performance metrics
  - Models
  - Architectures

But first we need to finish a bit of consistency!

DistSys Challenges

- **Heterogeneity**
- Openness
- Security
- Failure Handling
- Concurrency
- **Quality of Service**
- **Scalability**
- Transparency

Performance Challenges

PROBLEM

- Amazon: 100 ms extra latency costs 1% in sales , In 2020, Amazon Web Services (AWS) generated revenues of 45.37 billion U.S. dollars with its cloud services.
- Bing: 2s slowdown would reduce the revenue by 4.3%
- Google: (August 16 2013) 5 minute down time cost 545k$
- Increasing 400ms web search latency caused a decrease of about 0.74% in Google's search frequency

"Let's explore how global-scale companies tackle performance, reliability, and availability in production."

Modern Case Studies in Performance Engineering

- Performance is not just an academic metric—it's a core business driver at scale

Company	Practice	Impact
Google	**measurement-driven**-SLIs, SLOs, Error Budgets (SRE model)	Trade-offs between reliability and innovation.
Netflix	**failure-driven**-Chaos Engineering, Regional Failover Testing	Ensures system resilience under failure conditions.
Amazon	**latency-driven**-Latency Budgets, Availability Zones	Every 100ms of latency = 1% loss in revenue.

The Tail at Scale

Why p99 is the median user experience in a microservice world

Setup

- Your backend service has p99 latency = 100 ms.
- Sounds great — only 1% of requests are "slow."
- A user request fans out to 100 parallel backends.
- The user waits for the slowest reply.

P(all 100 finish < 100ms)

$$= 0.99^{100} \approx 0.37$$

63%

of users experience the "1% slow case"

SLI · SLO · SLA · Error Budget

The vocabulary the industry actually uses (Google SRE)

SLI

What you measure

A quantitative signal: request latency, availability, error rate, durability.

e.g., % of checkout calls returning 2xx within 500 ms

SLO

Target you commit to

A threshold on an SLI over a window — internal, aspirational.

e.g., 99.9% of checkouts < 500 ms over 28 days

SLA

Contract with teeth

External agreement with customers; breach triggers credits or penalties.

e.g., 99.5% → service credits if missed

EB

Error budget

100% − SLO. The failure you are allowed to spend on velocity / risk.

e.g., 99.9% ⇒ ~43 min/month to "spend"

The chain: You measure (SLI), you promise yourself (SLO), you promise others (SLA). Unspent error budget funds innovation; overspent budget freezes releases.

What is Performance?

- Marriam-webster:
  - the execution of an action
  - the fulfillment of a claim, promise, or request
  - the manner of reacting to stimuli

- Wikipedia
  - In computing, computer performance is the amount of useful work accomplished by a computer system. Outside of specific contexts, computer performance is estimated in terms of accuracy, efficiency and speed of executing computer program instructions.

What is Performance?

- Performance considers:
  - Latency (transmission delay)
  - Bandwidth (maximal transmission capacity)
  - Throughput (average transmission rate)
  - Response time (time to see result of action)

What is the first step???

How can we choose the best service as a client?

How can we make sure that we have provide the best service as a provider?

How can we understand that what are we going to design as an engineer?

QoS & SLA

Service Level Agreements (SLA)

- Service Level Agreements are fundamental to an effective cloud utilization and especially business customers need them to ensure risks and service qualities are prevented respectively provided in the way they want.
- The confirmed SLAs serve as a basis for compliance and monitoring of the QoS.
- Due to the dynamic cloud character, the QoS attributes must be monitored and managed consistently

How can I describe and measure the QoS?

KPI

How can we define SLA

- NIST has pointed out the necessity of SLAs, SLA management, definition of contracts, orientation of monitoring on Service Level Objectives (SLOs) and how to enforce them. (https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf)

- There are two major specification for describing SLAs
  - Web Service Level Agreement (WSLA) Language Specification, was developed by IBM with the focus on performance and availability metrics.
  - WS-Agreement (WS-A) was developed by the Open Grid Forum in 2007.
  - The newest update, which is based on the work of the European SLA@SOI project.

To measure service level objectives effectively, we need a more granular look at real-world performance data

Averages Lie — Think in Percentiles

If I tell you my service has an average latency of 80 milliseconds, how happy should you be?

The same service, two lenses

Metric	Value	Reading
Mean latency	80 ms	Looks healthy.
p50 (median)	60 ms	Typical user.
p95	220 ms	1 in 20 users.
p99	2,500 ms	1 in 100 -> complaints start.
p99.9	8,100 ms	Power users, bots, retries.

Two rules

1. **Never average percentiles.**
   Aggregate the raw distribution (or use HDR histograms / t-digest), then re-compute. Avg(p99) is mathematically meaningless.
   If Service A has p99 = 100 ms and Service B has p99 = 200 ms, the p99 of A + B is NOT 150 ms. That math is meaningless. You have to go back to the raw data and recompute.

2. **Optimize the tail, not the mean.**
   In fan-out architectures, overall p99 is dominated by the slowest dependency's tail. A 2× mean improvement on one service may not move user-visible latency at all.

Instrument for **histograms**, not just averages. Prometheus histogram / summary, OpenTelemetry Metrics. Store enough resolution that p99 and p99.9 are trustworthy.

Modern Performance Metrics Beyond Averages

- **Averages lie**—percentile-based metrics reveal performance outliers that affect real users.
- **Saturation** helps monitor system "pressure"—high CPU, memory, or queue depths.
- **Error Rates** (especially 5xx) are critical—too many might trigger circuit breakers or autoscaling.
- **Apdex** gives a user-centric performance score. Apdex > 0.85 is considered "excellent" service.

Metric	Description	Why It Matters
P95/P99/P999	Percentile latencies	Capture long-tail user experiences.
Saturation	Resource exhaustion level	Predicts overload before failure.
Error Rates	4xx/5xx, retry storms, fallbacks	Indicates service health.
Apdex Score	0–1 rating of user satisfaction	Combines response time + satisfaction thresholds.

How Systems Really Fail Under Load

When a distributed system goes down at 3 AM, it's almost never because one server exploded. It's because something much more interesting happened.

feedback loop ↻ load amplifies itself

Common triggers

- Slow downstream API / DB query
- Lock contention, hot keys
- GC pause, cold cache, noisy neighbor
- Gray failure: node slow, not dead — health checks pass

Mitigations

- Timeouts + bounded retries + jitter
- Circuit breakers, bulkheads, load shedding
- Backpressure end-to-end (not just at edges)
- Autoscale on saturation, not just CPU average

Little's Law

Imagine a coffee shop. Customers arrive at 2 per minute. Each customer spends 5 minutes inside, ordering, waiting, drinking. On average, how many customers are inside the shop at any given moment?

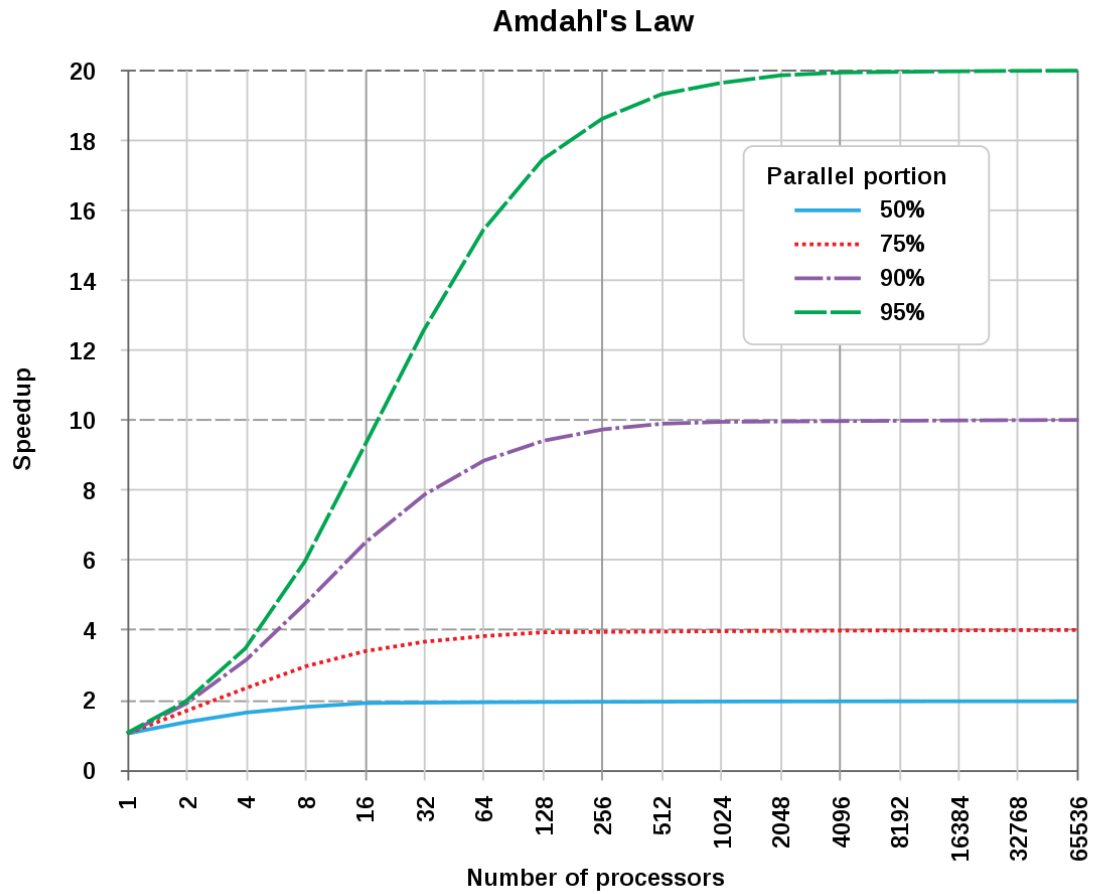
$$L = \lambda \times W$$

L	**Concurrent requests**	Average number in the system
λ	**Arrival rate**	Requests per second
W	**Response time**	Average time in system (sec)

Worked example. Service at **λ = 1,000 req/s** with average **W = 200 ms** ⇒ **L = 200** concurrent in flight. You need at least 200 threads / connections / goroutines - regardless of the latency distribution.

AMDAHLS LAW

- How will our maximum throughput change if we add a database
- In general terms, **Amdahl's Law** states that in parallelization, if **P** is the proportion of a system or program that can be made parallel, and **1-P** is the proportion that remains serial, then the maximum speedup **S(N)** that can be achieved using N processors is:
  - **S(N)=1/((1-P)+(P/N))**
- **If 95% of processing is in the web server (parralelized), then max speedup is 1/1-0.95 = 20X**



Amdahl Is Optimistic - Meet the USL

Amdahl's Law

$$S(N) = 1 \,/\, ((1-P) + P/N)$$

Models **serial work only**.
Asymptote: 1/(1−P). Throughput monotonically rises toward a ceiling.

Assumes zero coordination cost between workers — convenient for a single multicore CPU, wrong for a distributed system.

Universal Scalability Law (Gunther)

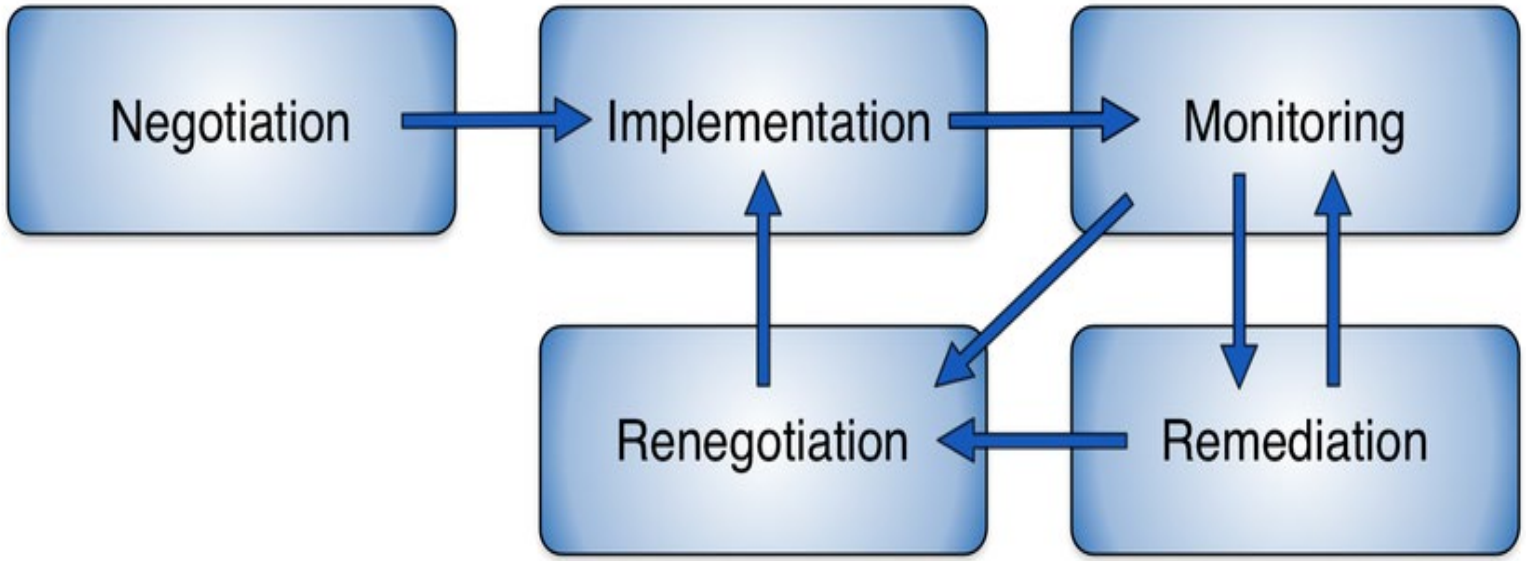
$$X(N) = N \,/\, (1 + \sigma(N-1) + \kappa \cdot N(N-1))$$

- σ — contention / serialization share
- κ — coherence / coordination cost

If κ > 0, throughput peaks at $N^* = \sqrt{((1-\sigma)/\kappa)}$ and then DECREASES.

SLA Life cycle

An SLA is not a document you sign once. it's a **living feedback loop**. The same loop students have been studying all semester in other forms: measure → react → adjust. It's monitoring and autoscaling. It's MAPE-K in autonomic computing. It's even the SRE error-budget workflow from two slides ago.



Service Level Objectives

- Service Level Objectives (SLOs) are a central element of every service level agreements (SLA), which include:
  - negotiated service qualities (service level)
  - corresponding Key Performance Indicators.
- SLOs contain the specific and measurable properties of the service, such as ***availability***, ***throughput or response time*** and often consist of ***combined or composed attributes***

SLO Characteristics

- SLOs should thereby have the following characteristics:
  - Repeatable
  - Measurable
  - Understandable
  - Significant
  - Controllable
  - Affordable
  - Mutually acceptable
  - Influential

We need KPI

SLO Characteristics

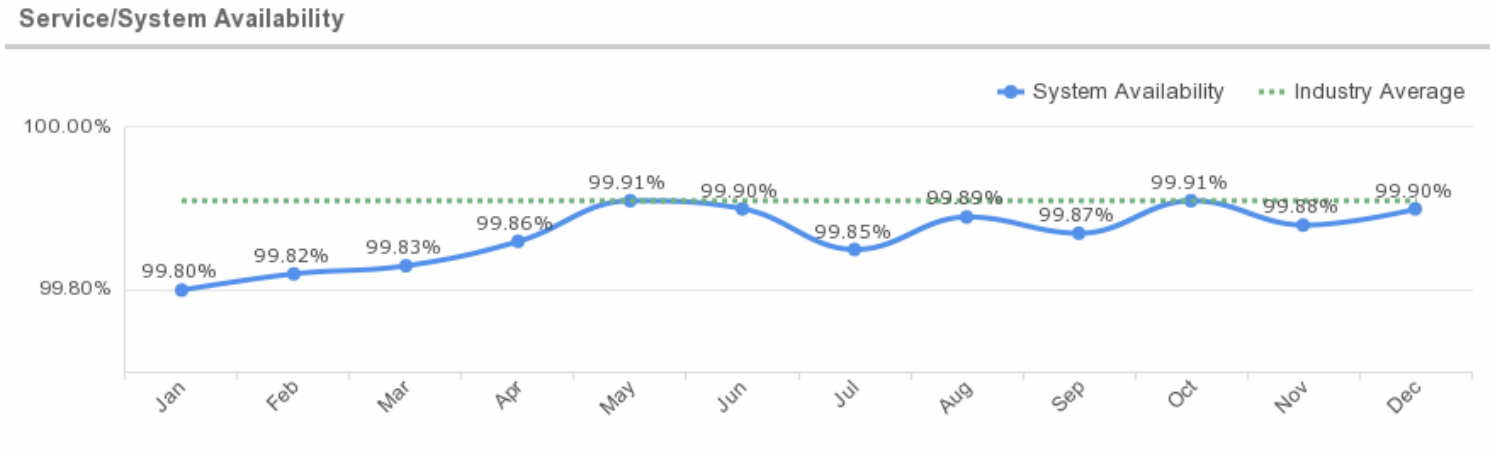
- A valid SLO specification might, for instance, look like this:
  - *The IT system should achieve an availability of 98% over the measurement period of one month. The availability represents thereby the ratio of the time in which the service works with a response time of less than 100ms plus the planned downtime to the total service time, measured at the server itself.*

KEY PERFORMANCE METRICS

- General Service KPIs
- Network Service KPIs
- Cloud Storage KPIs
- Backup and Restore KPIs
- Infrastructure as a Service KPIs

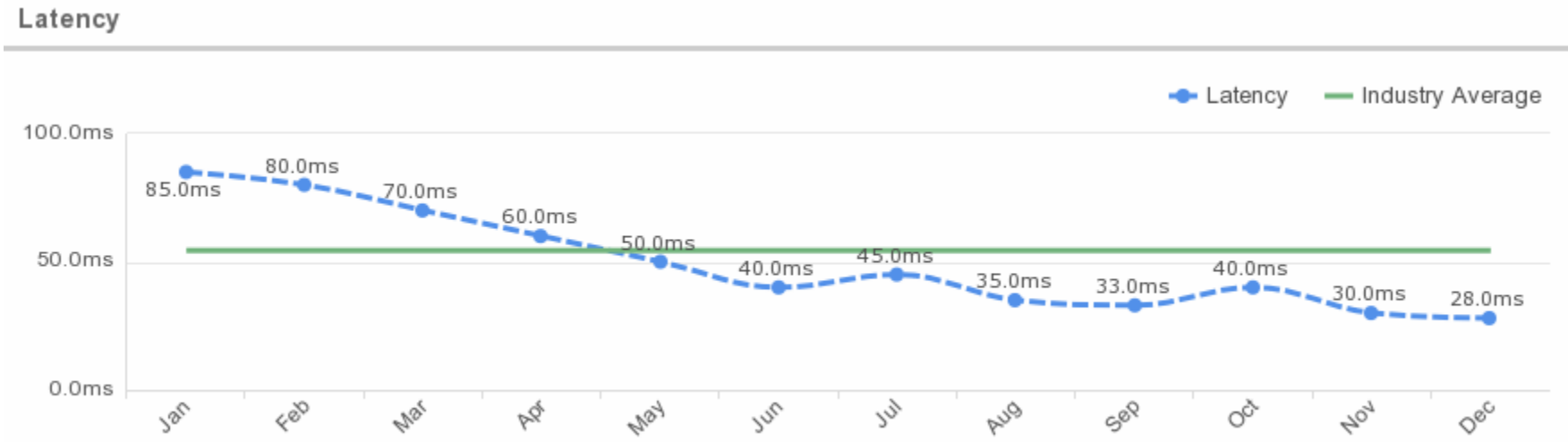
General Service KPIs

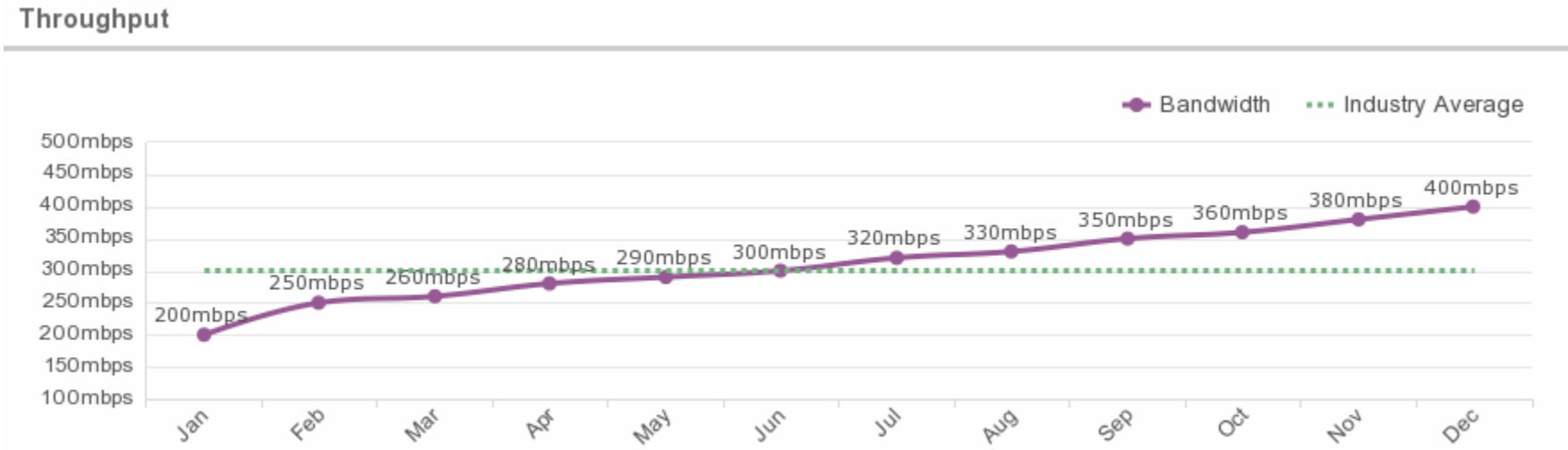
- Basic Services
- Security
- Service and Helpdesk
- Monitoring
- Etc.



Network Service KPIs

- Round Trip Time
- Response Time
- Packet Loss
- Bandwidth
- Throughput
- Network Utilization
- Latency
- Etc.





Modern Tools for Performance Benchmarking and Testing

- These tools help simulate real user behavior and stress scenarios before going live.
- **k6** and **Locust** are increasingly favored in CI pipelines for their scriptability.
- **Chaos Monkey** represents a shift toward proactive resilience testing.
- Monitoring isn't just about backend metrics. **Synthetic** (simulated) and **Real User Monitoring** give frontend visibility.

Tool	Use Case	Language
JMeter	Load test web APIs, simulate users	XML / GUI
k6	Scripted load testing, CI/CD integration	JavaScript
Locust	Python-based scalable load tests	Python
Chaos Monkey	Random failure injection	N/A
Synthetic Monitoring	Simulated transactions, global agents	e.g., Datadog
Real User Monitoring (RUM)	Measures real interactions in-browser/app	e.g., New Relic

Which Tool, When?

BEFORE	PRE-LAUNCH	STEADY	REALITY	PROACTIVE
dev + CI	*stress test*	*in production*	*real users*	*break on purpose*
k6	**JMeter / Locust**	**Synthetic**	**RUM**	**Chaos Monkey**
Scripted regression in every pipeline	Simulate peak load before go-live	Probe from outside, catch drift	What users actually experience	Surface unknown unknowns

Scenario — which tool(s) do you reach for?

① Every PR must not regress checkout latency by > 5%.
② Black Friday in two weeks — can the cluster handle 10× traffic?
③ Users in Tokyo complain the site feels slow; US dashboards look fine.
④ We trust the system. Prove it survives a zone failure at 3 PM on a Tuesday.

Cloud Storage KPIs

- Response Time
- Throughput
- Average Read Speed
- Average Write Speed
- Random Input / Outputs per second (IOPS)
- Sequential Input / Outputs per second (IOPS)
- Free Disk Space
- Provisioning Type
- Average Provisioning Time

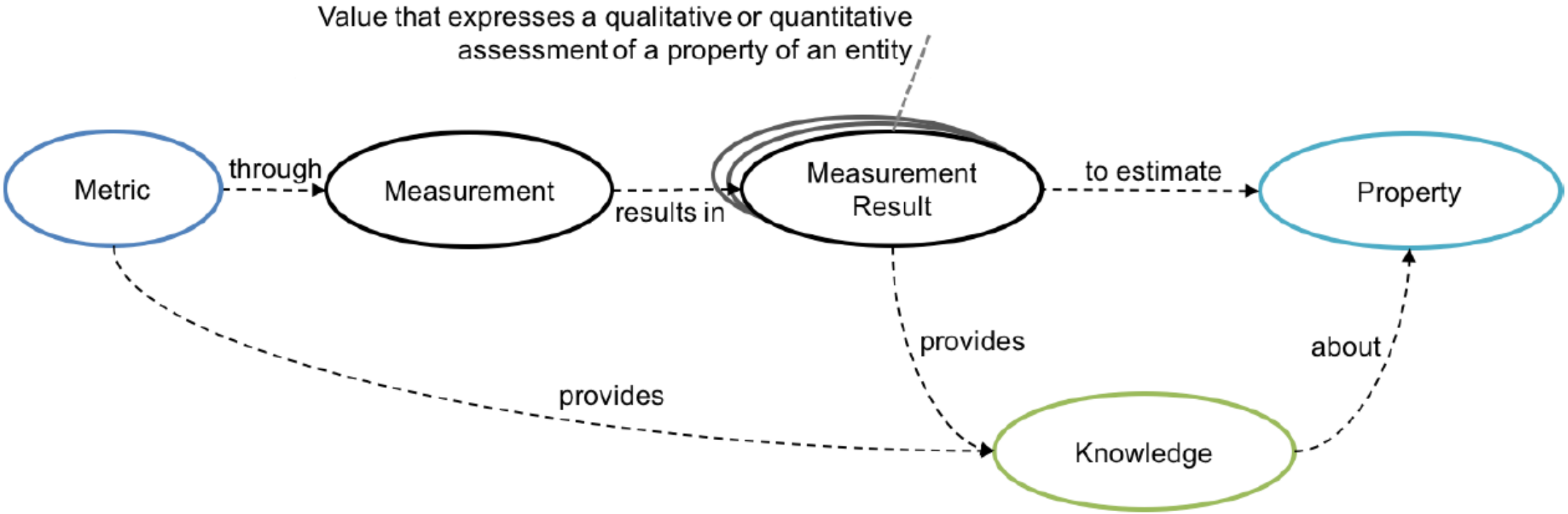
Backup and Restore KPIs

- Backup Interval
- Backup Type
- Time To Recovery
- Backup Media
- Backup Archive

Infrastructure as a Service KPIs

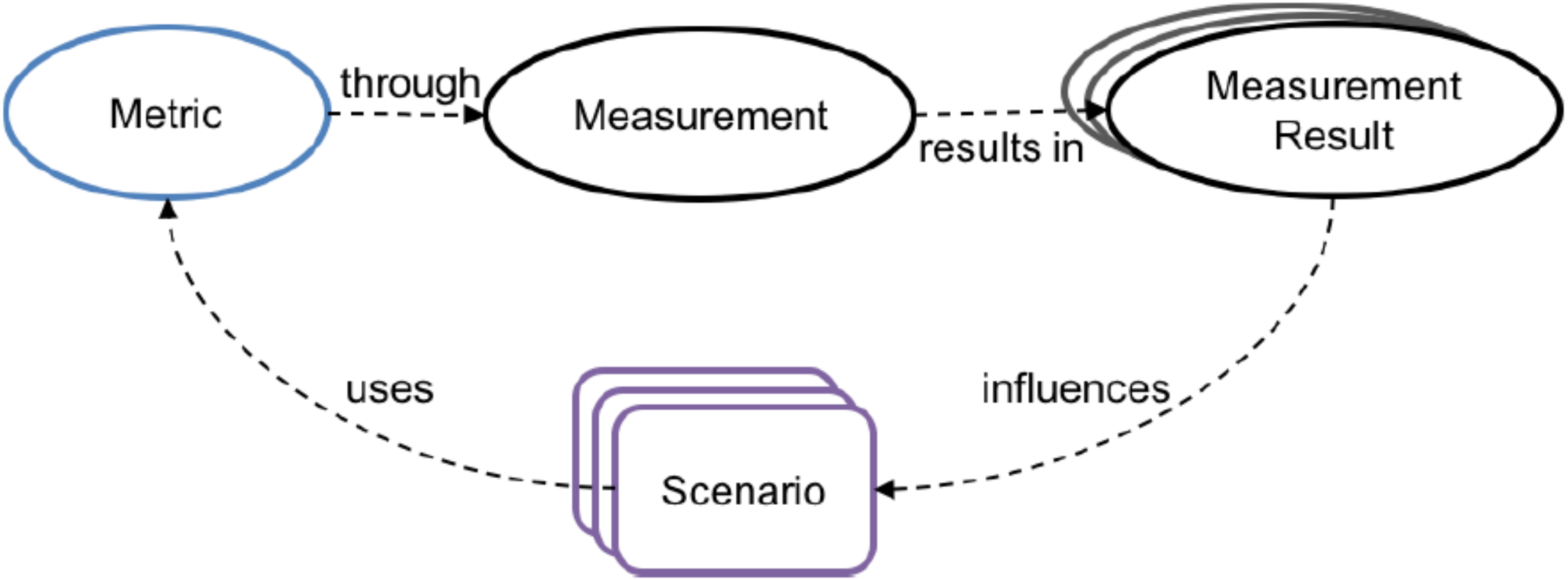
- VM CPUs
- CPU Utilization
- VM Memory
- Memory Utilization
- Minimum Number of VMs
- Migration Time
- Migration Interruption Time
- Logging

METRIC AND PROPERTY



https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500-307.pdf

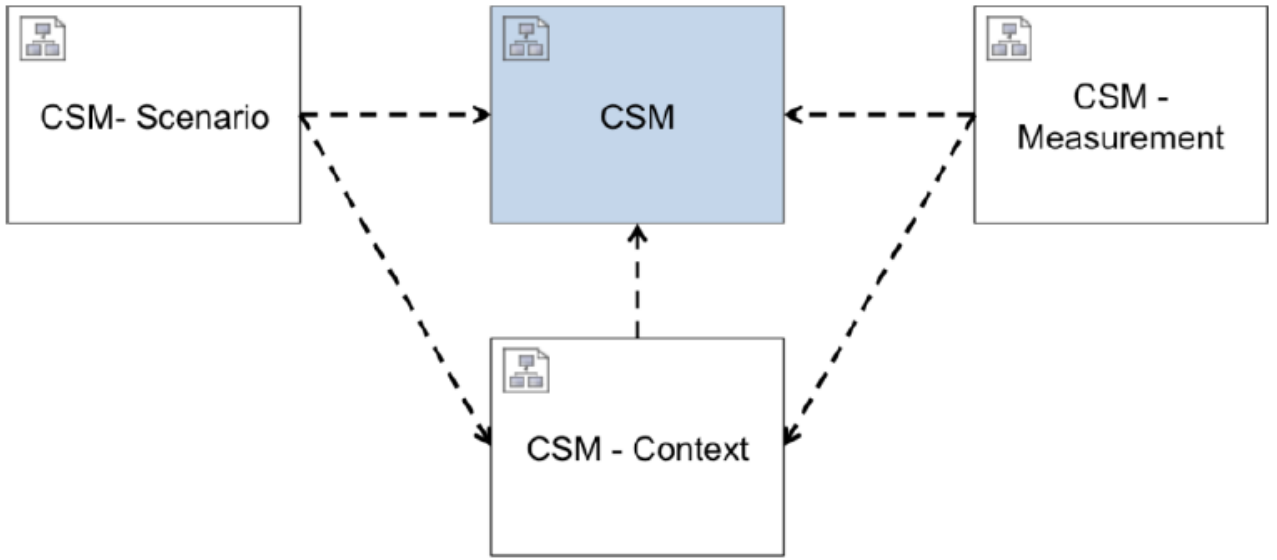
SCENARIO AND METRIC



https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500-307.pdf

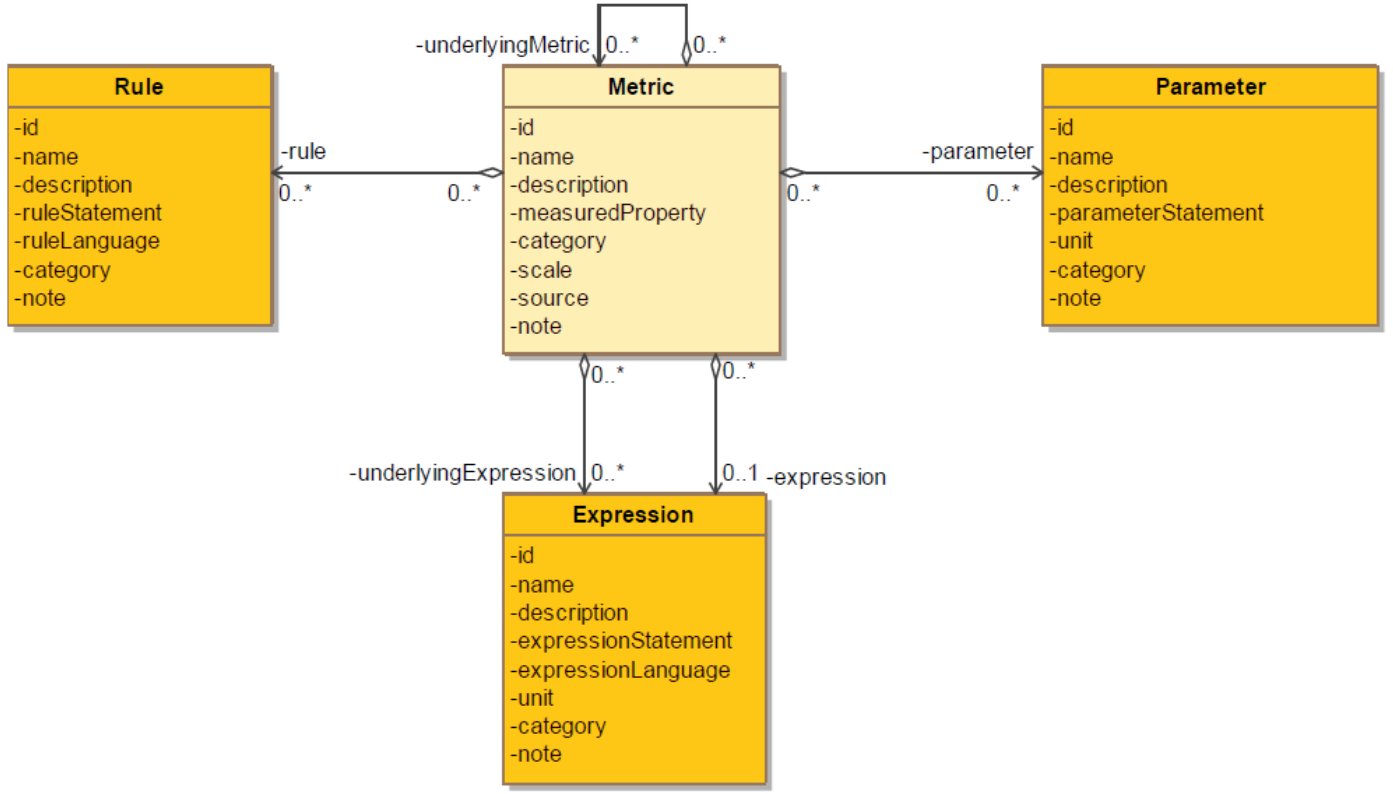
Cloud Service Metric Ecosystem Model

- CSM: The description and definition of a standard of measurement (e.g. metric for customer response time)
- CSM Context: The context related to using the CSM in a specific scenario. (e.g. objectives and applicability conditions of the customer response time metric)
- CSM Measurement: The use of the CSM to make measurements (e.g. the measurement of response time property based on the customer response time metric)
- CSM Scenario: The use of the CSM in a scenario (e.g. the selection and use of the customer response time metric in an SLA)



https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500-307.pdf

Cloud Service Metric (CSM)



https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500-307.pdf

Summary

- SLA: The agreement you make with the clients and users
- SLO: The objectives your team must hit to meet that agreement
- KPI: Key performance indicators

REFERENCES:

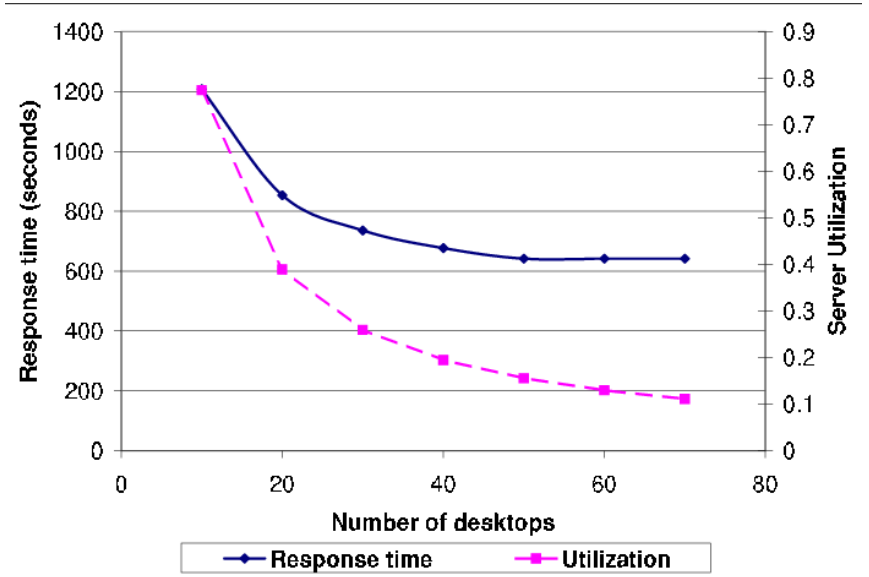
- http://www.thinkmind.org/articles/emerging_2013_3_30_40082.pdf
- https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500-307.pdf
- https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf
- https://www.cocop-spire.eu/content/key-performance-indicator-kpi-and-impact-evaluation-distributed-production-systems-%E2%80%93-importance-feedback
- https://www.atlassian.com/incident-management/kpis
- https://www.atlassian.com/incident-management/kpis/sla-vs-slo-vs-sli
- https://cloud.google.com/blog/products/gcp/sre-fundamentals-slis-slas-and-slos
- https://cloud.google.com/blog/products/gcp/availability-part-deux-CRE-life-lessons

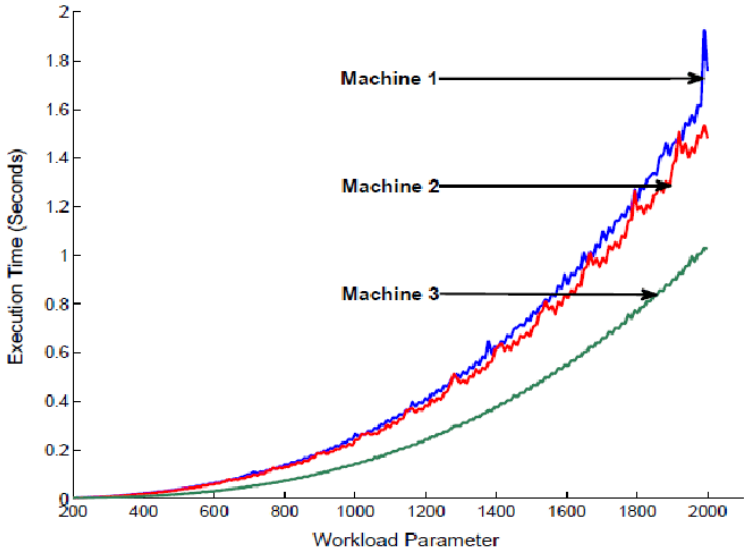
Prof. Tim Wood & Prof. Roozbeh Haghnazar

Performance Modeling

Performance Models

- Measuring performance is not always enough
- We want to Predict these metrics in advance
  - How will response time change if my workload doubles
  - How much memory CPU do I need to get a target throughput

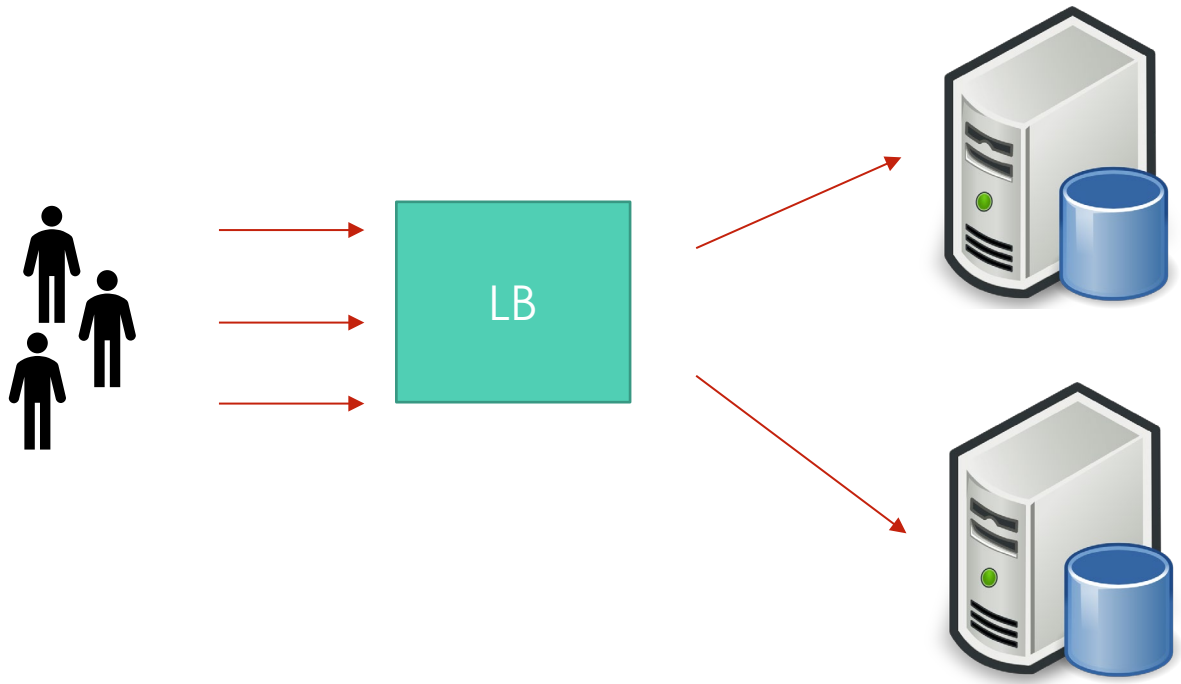




Model Scenario

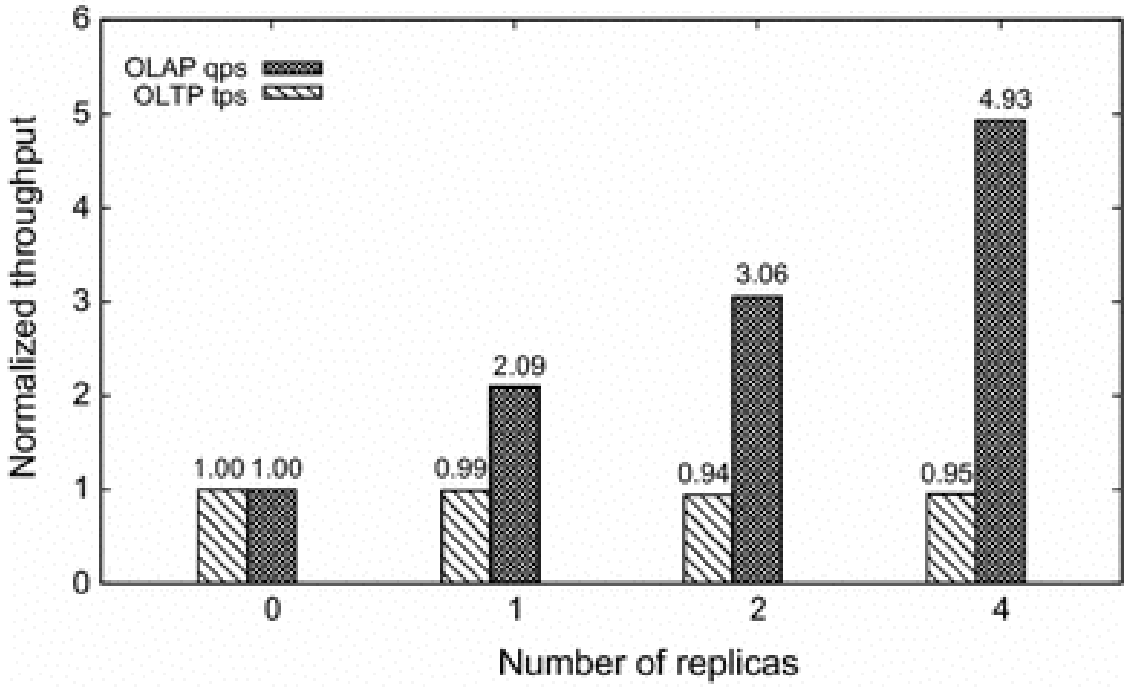
- Consider this web application

- What will affect its performance
  - Clients + Workload
  - Network + Latency
  - Application details
  - Server details
  - # Servers
  - LB Algorithms



THROUGHPUT MODELING

- How can we predict the maximum throughput **(Req/Sec)** of the system?
- Service Time □ 500 ms on webserver
- Throughput per replica ≈ $\frac{\#\ \mathrm{Th}reads}{Service\ time}$



Other Models

- Response Time
- Queuing Delay
- Queuing theory
- Performance Models
- ML Models